

RAPPORT

Dépôt : https://github.com/evradchris0-tech/projet_Docker_UTT

Mesures prises le 4 octobre 2026 sur ma machine (Windows 11, Docker Desktop 4.92.0, Docker Engine 29.8.0, Compose 5.5.1). Celles des questions 2, 3 et 4 viennent des commandes de `scripts/mesures.sh`.

1. Image de base

J'ai retenu `python:3.13.7-slim-bookworm` : image officielle, tag figé sur la version de Python et sur celle de Debian.

Je n'ai pas pris la variante complète (`python:3.13.7-bookworm`) parce qu'elle ajoute les compilateurs et les en-têtes de développement, qui servent à compiler des dépendances. Ici pip n'a rien compilé : le log du build montre 16 paquets, tous téléchargés en wheels.

Je n'ai pas pris alpine parce qu'Alpine utilise musl au lieu de glibc. Les trois paquets compilés du projet (`greenlet`, `pydantic_core`, `sqlalchemy`) ont été téléchargés en wheels `manylinux`, faits pour glibc. Sur Alpine, pip dépend de l'existence de wheels `musllinux` pour chaque paquet, sinon il compile depuis les sources. De plus `useradd`, que mon Dockerfile utilise, n'existe pas dans Alpine de base. Je n'ai pas testé cette variante.

2. Cache

Mes instructions vont de la plus stable à la plus volatile : `FROM`, `ENV`, `WORKDIR`, `RUN useradd`, `COPY requirements.txt .`, `RUN pip install -r requirements.txt`, `COPY --chown=appuser:appuser app/ ./app/`, puis `USER`, `EXPOSE`, `HEALTHCHECK` et `CMD`.

Une couche est réutilisée si son instruction et ses fichiers d'entrée n'ont pas changé, et si toutes les couches précédentes ont été réutilisées. Quand je modifie une ligne de code, seul le `COPY` de `app/` est refait : `requirements.txt` n'a pas changé, donc `pip install` reste en cache. Les quatre instructions qui suivent ne coûtent rien, ce sont des métadonnées (0B dans `docker history`).

Durées relevées dans la sortie de `docker compose build --progress=plain web` :

Étape	Premier build (<code>--no-cache</code>)	Second build (une ligne ajoutée dans <code>app/main.py</code>)
[3/6] <code>RUN useradd --create-home --uid 10001 appuser</code>	3,8 s	CACHED
[4/6] <code>COPY requirements.txt .</code>	1,5 s	CACHED
[5/6] <code>RUN pip install -r requirements.txt</code>	16,4 s	CACHED
[6/6] <code>COPY --chown=appuser:appuser app/ ./app/</code>	2,2 s	3,0 s
Durée totale (<code>time</code>)	47,8 s	16,4 s

Le build passe de 48 s à 16 s. Si `COPY app/` était placé avant `pip install`, chaque modification du code relancerait les 16 s d'installation.

En développement, Compose Watch (`develop.watch` dans `compose.yaml`) évite même ce rebuild : avec `docker compose up --watch`, un fichier modifié dans `app/` est copié dans le conteneur, puis `web` redémarre ; seule une modification de `requirements.txt` reconstruit l'image. C'est la raison du `--chown` : sans lui, `/app/app` appartient à `root` alors que le conteneur tourne en `appuser`, et la suppression d'un fichier n'était pas répercutée (`rm: cannot remove '/app/app/__init__.py': Permission denied`).

3. Taille

IMAGE	ID	DISK USAGE	CONTENT SIZE
<code>todoist-web:1.0</code>	<code>9fce56a33041</code>	238MB	56.8MB

D'après `docker history todoist-web:1.0`, l'image de base pèse 136,5 Mo (Debian bookworm 85,2 Mo, paquets système 10,4 Mo, Python 40,9 Mo), `RUN pip install` 44,5 Mo, `COPY app/` 77,8 ko, et `useradd`, `requirements.txt` et `WORKDIR` moins de 100 ko. Les couches font 181 Mo une fois décompressées. `CONTENT SIZE` (56,8 Mo) est la

taille compressée, celle qui transite lors d'un `push` ou d'un `pull`. `DISK USAGE` (238 Mo) est la somme des deux.

Je n'ai pas de mesure avant et après : l'image a été construite d'emblée sur `slim`, avec `PIP_NO_CACHE_DIR=1` pour ne pas garder le cache de pip dans la couche, et un `.dockerignore` qui écarte les scripts, les mesures et `.env`.

4. Persistance

MySQL écrit ses données dans `/var/lib/mysql`, où `compose.yaml` monte le volume nommé `db_data` (nom réel : `todoist_db_data`). Les données sont donc dans le volume, pas dans le conteneur.

`docker compose down` supprime les trois conteneurs et le réseau `todoist_default`, mais pas le volume. Au `up` suivant, Compose le recrée et remonte le même volume. MySQL trouve un dossier de données déjà rempli : il ne réinitialise rien et ignore les variables `MYSQL_*`. `web` et `adminer` attendent que `db` soit `healthy`, puis `create_all` ne crée aucune table puisqu'elles existent. Mesuré : 2 tâches au départ, 3 après un ajout, 3 après `docker compose restart`, 3 après `down` puis `up`. Après le `down`, `docker volume ls` affichait toujours `todoist_db_data`.

`docker compose down -v` supprime en plus le volume. Au `up` suivant, Compose crée un volume vide, MySQL refait toute son initialisation (base et utilisateur créés à partir de `.env`), et l'application recrée ses tables et le projet « Boîte de réception ». Mesuré sur une copie du projet lancée sous un autre nom (`-p todoist-verif`) pour ne pas effacer ma base : 1 tâche, puis `down -v` affiche `Volume todoist-verif_db_data Removed`, puis `up` rend 0 tâche. Ce `up` a pris 2 min 14 s, contre 28 s après un simple `down`.

5. Difficulté

Pour vérifier que le projet démarre chez quelqu'un d'autre, j'ai copié le dépôt dans un dossier vide, fait `cp .env.example .env`, puis lancé `docker compose -p todoist-verif up -d --build --wait`. Après 2 min 13 s :

```
dependency failed to start: container todoist-verif-db-1 is unhealthy
```

`web` n'avait pas démarré, alors que dans mon dossier de travail tout fonctionnait. Diagnostic :

1. `docker compose ps` montrait `db` en `Up About a minute (unhealthy)`. Le conteneur tournait : ce n'était pas un plantage de MySQL, mais un verdict du healthcheck.
2. `docker logs --timestamps todoist-verif-db-1` : l'initialisation de MySQL a duré 2 min 35 s (de 00:24:36 à 00:27:11).
3. `docker inspect -f '{{.State.Health.Status}}'` toutes les 5 s : `starting` jusqu'à 118 s, `unhealthy` à 123 s, `healthy` à 161 s.
4. Mon healthcheck laissait `start_period: 20s` plus 20 essais espacés de 5 s, soit 120 s.

La cause : sur un volume vide, MySQL met plus de temps à s'initialiser que le délai accordé par mon healthcheck. `db` est déclarée `unhealthy`, et comme `web` dépend de `db` avec `condition: service_healthy`, Compose abandonne son démarrage. Je ne le voyais pas chez moi parce que mon volume existait déjà et que MySQL démarre alors en moins de 30 s.

Correction : `start_period: 300s` dans `compose.yaml`. Pendant ce délai, les échecs ne sont pas comptés et le premier succès rend le service `healthy` tout de suite, donc rien n'est ralenti quand MySQL est rapide. Vérification : le même test réussit dès la première commande, les trois services sont `healthy` après 6 min 38 s (construction de l'image comprise) et `GET /` répond 200.

Ce que j'en retiens : tester sur un volume vide, pas seulement sur ma machine où tout est déjà initialisé.